

Learned Page Replacement under a Workload Shift

A Comparison with FIFO, LRU, and Belady's Optimal Policy

CSE-307: Operating Systems - Part B, Track 1

Fahim Azmul Hasan | Student ID: 2024-14014

Section A | Level 3, Term 1

Submitted to Lecturer Khaled Hasan Irfan

3 October 2026

Abstract

Page replacement decisions become difficult when a program's access pattern changes. This study implements FIFO, least recently used (LRU), Belady's optimal policy (OPT), and a small decision-tree eviction policy. Each evaluation trace contains 6,000 references to 64 pages, with a shift after reference 3,000 from a sequential hot set to either uniform random accesses or rotating bursts. A tree trained only on separate locality traces uses recency and frequency to score resident pages. Across ten evaluation seeds and four frame capacities, all policies receive the same traces and retain their state across the shift. With 16 frames, LRU and the learned policy attain 91.11% and 91.14% hit ratios before the shift. After a random shift, their hit ratios fall to 25.07% and 24.53%; after a bursty shift, they attain 78.65% and 70.06%. The learned policy does not provide a reliable improvement over LRU. Its greater degradation on bursts illustrates the risk of carrying a learned locality rule into a changed workload. These are reproducible simulator measurements, rather than measurements of Windows paging or VM performance.

1. Introduction

When a requested page is absent from the available frames, a page-replacement simulator records a fault. Once the frames are full, a policy must choose a resident page to remove. This models the central replacement decision in memory management, while leaving out details such as page-fault service time and disk writes [1]. A good eviction choice preserves pages that will be reused soon.

This paper asks whether a lightweight classifier can make useful eviction choices from historical access features, and whether those choices remain useful after a workload shift. The comparison includes FIFO and LRU as online baselines, and OPT as a full-trace reference with future knowledge. The primary outcomes are page-fault count and hit ratio, reported separately for the two halves of each trace.

The implementation deliberately remains small: Python generates synthetic traces, runs four replacement policies, and writes CSV results and figures. No operating system modification, virtual machine, application interface, or large learning model is required. The contribution is a controlled experiment linking capacity, locality, and distribution shift to observed replacement behavior.

2. Classical and Learned Policies

2.1 Classical implementations

All policies begin with empty frames. An absent page is loaded immediately; eviction occurs only when a fault finds all frames occupied. FIFO removes the resident page with the earliest insertion index, which is unchanged by a hit. LRU removes the resident page with the oldest last-access index, updated on every reference. OPT removes a page whose next reference lies furthest in the future; a page with no remaining reference is preferred. These definitions follow standard treatments of page replacement [2].

The implementation uses resident-page sets and timestamp dictionaries. On an eviction it scans the resident candidates. OPT first scans the trace backwards to compute next-use positions. Deterministic page-ID tie breaking makes repeated runs reproducible. OPT can use both halves of an evaluation trace, so it is an offline benchmark. Its total faults provide a lower bound for that trace; separate phase counts are descriptive and need not each be a phase-specific minimum.

2.2 A classifier for eviction candidates

At each full-cache fault, every resident page is represented by three features:

Feature	Definition before current access
Recency	Number of references since the page's most recent access.
Recent frequency	Access count in the preceding 64 references.
Total frequency	Access count over the entire preceding trace history.

Features are maintained for all previously accessed pages, including evicted pages. Neither the page ID nor the phase label is a model input.

Training replays eight independent locality traces under LRU at 8, 16, and 24 frames. At each eviction opportunity, the future of that training trace labels every furthest-next-use candidate as class 1 and all other resident pages as class 0. This is supervision on LRU-generated states, not a replay of OPT's cache states. Multiple candidates can receive class 1 when they have no future access.

A scikit-learn decision tree uses maximum depth 5, minimum leaf size 30, balanced class weights, and random seed 307. Depth and leaf-size restrictions limit model complexity; trees produce interpretable feature-based rules [3]. At evaluation time the resident page with the highest class-1 score is evicted. Equal scores fall back to LRU, then the smallest page ID.

The tree is trained once and frozen. Its input statistics change online, but its parameters are not retrained after the shift. Future accesses are used only for training labels and OPT. The learned policy itself receives historical features only. Learning-augmented caching research motivates such a comparison, but this small classifier does not implement a robust competitive caching algorithm [4].

3. Experimental Setup and Validation

3.1 Workload and controls

Both scenarios begin with 3,000 locality-heavy references. With probability 0.9, the generator selects the next page in a repeating sequence through pages 0-7; otherwise it selects uniformly from pages 8-63. The sequential cursor advances only on a hot-set reference. Thus the trace combines an eight-page recurring hot set with occasional background accesses.

At the boundary, the second half changes to one of two patterns. In the random scenario, every page is selected independently and uniformly from all 64 pages. In the bursty scenario, an eight-page hot set rotates every 128 references through eight disjoint blocks. Each reference selects uniformly from the current hot set with probability 0.8, or from all 64 pages with probability 0.2. Background draws may also select a hot-set page. The shift is abrupt and prescribed; the policies are not told when it occurs.

Setting	Value
Trace length / shift index	6,000 references / 3,000 (zero based)
Page universe / hot-set size	64 / 8 pages
Frame capacities	8, 12, 16, 24
Evaluation seeds	100-109, for each scenario
Training seeds / length	0-7 / 3,000 locality references each
Candidate training rows	189,312; 25,183 labelled class 1
Fitted tree	Depth 5; 31 leaves
Evaluation policy runs	2 x 10 x 4 x 4=320

For a given scenario and seed, each policy and capacity receives exactly the same saved trace. The first halves of the paired random and bursty scenarios are also identical. Frames, timestamps, and frequency histories are retained across the boundary. There is no reset that would hide the cost of adapting to the new pattern. Model settings were fixed before interpreting evaluation outcomes; no test-trace labels or results are used to fit or select the tree.

3.2 Metrics and reproducibility

For each 3,000-reference phase, the hit ratio is $H=1-F/3000$, where F is the number of faults. Results are averaged across ten seeds. Standard deviations describe between-trace variation, rather than uncertainty across individual references. Changes in hit ratio are expressed in percentage points. All faults, including initial cold-start faults, are counted. There are 1,920,000 recorded policy-reference outcomes. Separate 250-reference windows summarize the trajectory.

Execution used Windows 11 and Python 3.12.14, with NumPy 2.3.5, scikit-learn 1.9.1, and Matplotlib 3.11.2. The repository preserves evaluation traces, SHA-256 hashes, compressed training candidates, per-access fault flags, per-seed metrics, aggregate tables, and the fitted tree's readable rules. Windows supplies the Python runtime; it does not supply the simulated fault measurements.

Validation includes the known FIFO anomaly trace (9 faults with three frames and 10 with four), an LRU example, repeated-page and empty-trace checks, and 240 short trace/capacity comparisons of OPT against an independent exhaustive solver. All six test methods passed, including learned-policy tie breaking and independence of prefix decisions from future accesses. These checks establish behavior on controlled cases; they do not establish that synthetic traces represent all real programs.

4. Results at 16 Frames

Table 1 reports phase-level means. Mean faults can be fractional because they are averaged over ten traces. After-shift standard deviations are shown in percentage points. Before-shift standard deviations are 0.83 for FIFO, 0.45 for LRU, 0.27 for OPT, and 0.52 for Learned; the paired scenarios share this same first-half workload.

Table 1. Measured results with 16 frames; 3,000 references per phase and ten seeds.

Scenario	Policy	Before faults	Before hit %	After faults	After hit %	After SD pp
Random	FIFO	475.2	84.16	2254.6	24.85	0.55
Random	LRU	266.7	91.11	2248.0	25.07	0.46
Random	OPT	190.6	93.65	1306.3	56.46	0.48
Random	Learned	265.8	91.14	2264.2	24.53	0.98
Bursty	FIFO	475.2	84.16	825.1	72.50	0.75
Bursty	LRU	266.7	91.11	640.6	78.65	0.65
Bursty	OPT	190.6	93.65	421.0	85.97	0.43
Bursty	Learned	265.8	91.14	898.3	70.06	0.78

Before the shift, FIFO records 475.2 faults, while LRU records 266.7. FIFO can discard a heavily reused page solely because it entered early; LRU protects the recently reused hot set more effectively. The learned policy records 265.8 faults, only 0.9 fewer than LRU on average. This 0.03-point hit-ratio difference is too small to support a strong improvement claim in this experiment. OPT records 190.6 faults, showing remaining scope beyond historical heuristics.

After the random shift, the online policies converge near a 25% hit ratio. For independent uniform references to 64 pages and a full cache of 16 distinct pages, the next reference hits the cache with probability $16/64=0.25$, regardless of how a history-only policy selected those pages. This explains why even a learned rule cannot reliably recover the earlier hit rate. Finite traces produce small deviations around that expectation.

OPT retains a 56.46% hit ratio after the random shift because it knows the future. This does not contradict the online expectation: OPT's resident set is selected using future references, rather than being independent of the next random draw. Its result should not be interpreted as a deployable online prediction accuracy.

The bursty scenario preserves useful short-term reuse. LRU records 640.6 faults and a 78.65% hit ratio, compared with 825.1 faults and 72.50% for FIFO. The learned policy instead records 898.3 faults and 70.06%, falling below both online baselines. Its gap to LRU is 8.59 percentage points, or 257.7 additional faults per phase on average. Thus retaining locality does not guarantee that a learned policy trained on a different locality structure will exploit it well.

For the random shift, the hit-ratio drops are 59.31 points for FIFO, 66.04 for LRU, 37.19 for OPT, and 66.61 for Learned. For the bursty shift, they are 11.66, 12.46, 7.68, and 21.08 points, respectively. Learned degrades the most by this absolute drop metric in both scenarios. A drop depends on the starting value, so it should be read alongside the actual post-shift fault counts.

5. Capacity Sensitivity and Interpretation

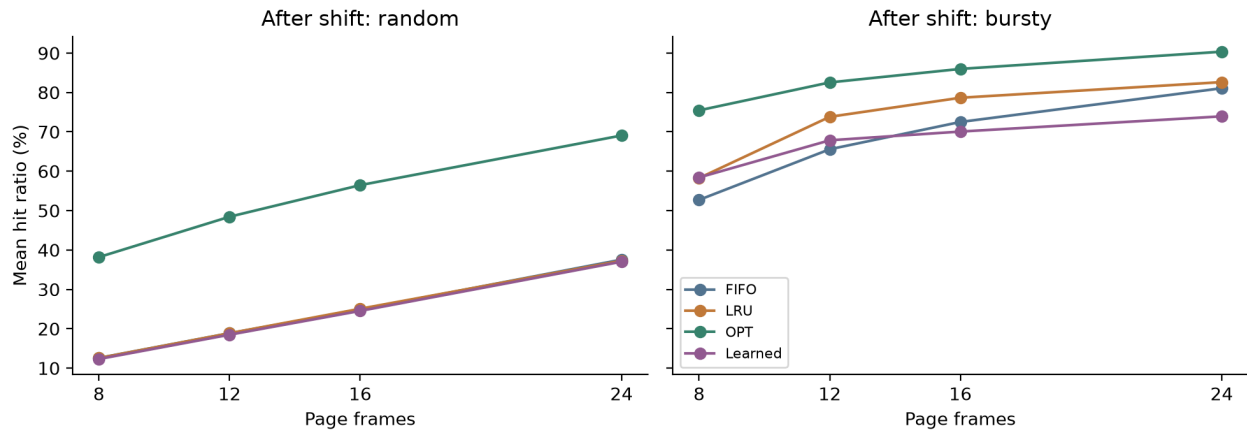


Figure 1. After-shift mean hit ratio versus frame capacity, across ten traces. The two panels share the same page universe and first-half generator.

Figure 1 extends the comparison to four capacities. Under random accesses, online hit ratios rise approximately with the fraction of the page universe that fits in memory: 12.5%, 18.75%, 25%, and 37.5% for 8, 12, 16, and 24 frames. These are expectations from the generator, not fitted predictions. OPT remains above the online policies because it can reserve space for nearer future requests. More frames reduce pressure but do not restore predictability.

Under bursts, extra frames help retain the current hot set along with some background pages. The learned policy's performance depends strongly on capacity; the resulting curve should be interpreted as evidence for these four tested settings, rather than as a universal monotonicity guarantee for a classifier-based policy. The complete phase and per-seed tables are retained in the results folder.

The training distribution has a persistent hot set. Its labels reward protecting pages that appear repeatedly and evicting infrequent background pages. In the bursty phase, formerly useful pages become stale when the active block rotates. Recent-frequency features eventually respond as the 64-reference window changes, but total frequency carries old history forward. The learned rule has no explicit concept of a block transition and no mechanism for relearning that transition.

The fitted tree assigns impurity-based feature importances of 83.92% to recent frequency, 10.86% to recency, and 5.22% to total frequency. These values describe how the training tree splits its data; they are not causal measures of which feature produced the post-shift faults. The poor burst result is consistent with a mismatch between training and evaluation states. Demonstrating a specific feature's causal role would require an ablation experiment, which this study does not perform.

There is a second mismatch: training candidates come from caches maintained by LRU, while evaluation candidates come from caches maintained by the learned rule. Earlier learned evictions can create resident sets that differ from the training states. The classifier then scores candidates under its own accumulated mistakes. This is an additional reason that a plausible supervised model may not produce a good sequential eviction policy.

These observations support a practical distinction between updating access statistics and adapting the model itself. The implementation does the former. It does not prove robustness to distribution shift. Work on learned caching warns that following predictions without safeguards can perform poorly [4]; the present experiment illustrates that concern in a small simulation, without claiming the theoretical guarantees of that work.

6. Limitations, Conclusion, and Reproduction

6.1 Limits of the evidence

The study uses two synthetic shift scenarios, one page universe, one trace length, and one fixed tree configuration. Ten evaluation seeds quantify variation within these generators; they do not represent ten different real applications. Training seeds are separate from evaluation seeds, but both are generated by the same first-half process. Generalization to file-system caches or real virtual-memory reference streams remains untested.

The simulator assumes equal-size pages, a fully associative resident set, and immediate loading after a fault. It omits dirty pages, write-back costs, prefetching, TLBs, multiple processes, fault-service latency, and VM scheduling. A recorded fault therefore represents a simulated miss, not a measured kernel fault or a disk I/O operation. Hit ratio alone cannot establish application speedup or lower energy use.

The tree's balanced-class scores are used only to rank eviction candidates. They are not calibrated correctness probabilities. No classifier-accuracy or confidence claim is made. Likewise, simulation wall-clock time is not a benchmark of OS performance or the cost of deployment. Learning introduces bookkeeping and inference work, but this experiment does not quantify that overhead separately.

6.2 Conclusion

The experiments show that the access pattern determines what page replacement can achieve. A repeating hot set favors LRU over FIFO. An abrupt switch to uniform random references removes the historical signal and pushes online hit ratios toward the capacity-to-universe fraction. Rotating bursts preserve locality, but require a policy to follow a moving working set.

The lightweight learned policy nearly matches LRU before the shift and performs worse on bursts afterward. Its strongest result is therefore not a performance win, but an observable failure to generalize a stable-locality rule. For these tested workloads, LRU is the stronger simple online choice. Possible extensions include time-decayed frequency, training on mixed workload families, collecting states from the learned policy, or a monitored fallback. Those are future experiments, not results claimed here.

6.3 Reproduction and assistance disclosure

Install the pinned Python dependencies, run `python -m unittest -v test_algorithms`, then `python experiment.py`. The script regenerates the raw traces, candidate labels, per-access outcomes, aggregate metrics, and three figures. Run `python build_paper.py` to regenerate this standalone LaTeX source from the aggregate CSV. AI assistance was used for code generation, the experimental workflow, and drafting this report. The reported measurements come from executed simulations. The submitting student must review the design, results, and analysis and be able to explain the implementation.

References

- [1] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, Operating Systems: Three Easy Pieces, ch. 22, "Beyond Physical Memory: Policies." <https://pages.cs.wisc.edu/remzi/OSTEP/vm-beyondphys-policy.pdf>.
- [2] Cornell University, CS 4410, "Lecture 15: Page replacement," Summer 2015. <https://www.cs.cornell.edu/courses/cs4410/2015su/lectures/lec15-replacement.html>.
- [3] scikit-learn developers, "Decision Trees," documentation. <https://scikit-learn.org/stable/modules/tree.html>. Accessed 3 Oct. 2026.
- [4] T. Lykouris and S. Vassilvitskii, "Competitive caching with machine learned advice," arXiv:1802.05399, 2018, revised 2020. <https://arxiv.org/abs/1802.05399>.